

Start coding or [generate](#) with AI.

```
import sqlite3
import pandas as pd

# Connect to an in-memory SQLite database
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

print("Database connection established.")
```

Database connection established.

▼ Creating Tables

```
# Schema for Students table
cursor.execute('''
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Email VARCHAR(100)
);
''')

# Schema for Courses table
cursor.execute('''
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100),
    Credits INT
);
''')

# Schema for Enrollments table
cursor.execute('''
CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
''')

print("Tables created successfully.")
```

Tables created successfully.

▼ Inserting Sample Data

```
cursor.execute("INSERT INTO Students (StudentID, FirstName, LastName, Email) VALUES (101, 'John', 'Doe', 'john.doe@example.com'), (102, 'Jane', 'Smith', 'jane.smith@example.com');")
cursor.execute("INSERT INTO Courses (CourseID, CourseName, Credits) VALUES (201, 'Introduction to SQL', 3), (202, 'Advanced SQL', 4);")
cursor.execute("INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, EnrollmentDate) VALUES (1, 101, 201, CURRENT_DATE);")

conn.commit()
print("Sample data inserted successfully.")
```

```
-----
ProgrammingError                                Traceback (most recent call last)
/tmp/ipython-input-3370581591.py in <cell line: 0>()
----> 1 cursor.execute("INSERT INTO Students (StudentID, FirstName, LastName, Email) VALUES (101, 'John', 'Doe', 'john.doe@example.com'), (102, 'Jane', 'Smith', 'jane.smith@example.com');")
      2 cursor.execute("INSERT INTO Courses (CourseID, CourseName, Credits) VALUES (201, 'Introduction to SQL', 3), (202, 'Advanced SQL', 4);")
      3 cursor.execute("INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, EnrollmentDate) VALUES (1, 101, 201, CURRENT_DATE);")
      4
      5 conn.commit()

ProgrammingError: Cannot operate on a closed database.
```

```
import sqlite3
import pandas as pd

# Connect to an in-memory SQLite database
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

print("Database connection established.")
```

Database connection established.

```
# Schema for Students table
cursor.execute('''
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Email VARCHAR(100)
);
''')

# Schema for Courses table
cursor.execute('''
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100),
    Credits INT
);
''')

# Schema for Enrollments table
cursor.execute('''
CREATE TABLE Enrollments (
```

```
EnrollmentID INT PRIMARY KEY,
StudentID INT,
CourseID INT,
EnrollmentDate DATE,
FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
'''
```

```
print("Tables created successfully.")
```

Tables created successfully.

```
cursor.execute("INSERT INTO Students (StudentID, FirstName, LastName, Email) VALUES (101, 'John', 'Doe', 'john.doe@example.com'), (102, 'Jane', 'Smith', 'jane.smith@example.com');")
cursor.execute("INSERT INTO Courses (CourseID, CourseName, Credits) VALUES (201, 'Introduction to SQL', 3), (202, 'Advanced SQL', 4);")
cursor.execute("INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, EnrollmentDate) VALUES (1, 101, 201, CURRENT_DATE);")
```

```
conn.commit()
print("Sample data inserted successfully.")
```

Sample data inserted successfully.

```
# Query to generate a list of students enrolled in a specific course
print("Students enrolled in CourseID 201:")
students_in_course_query = '''
SELECT s.FirstName, s.LastName
FROM Students s
JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.CourseID = 201;
'''

df_students_in_course = pd.read_sql_query(students_in_course_query, conn)
display(df_students_in_course)
```

Students enrolled in CourseID 201:

	FirstName	LastName
0	John	Doe

```
# Query to count total enrollments per course
print("Total enrollments per course:")
enrollments_per_course_query = '''
SELECT c.CourseName, COUNT(e.EnrollmentID) AS TotalEnrollments
FROM Courses c
LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
GROUP BY c.CourseName;
'''

df_enrollments_per_course = pd.read_sql_query(enrollments_per_course_query, conn)
display(df_enrollments_per_course)
```

Total enrollments per course:

	CourseName	TotalEnrollments
0	Advanced SQL	0
1	Introduction to SQL	1

```
# Close the connection
conn.close()
print("Database connection closed.")
```

Database connection closed.

⌵ Executing Queries

```
# Query to generate a list of students enrolled in a specific course
print("Students enrolled in CourseID 201:")
students_in_course_query = '''
SELECT s.FirstName, s.LastName
FROM Students s
JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.CourseID = 201;
'''

df_students_in_course = pd.read_sql_query(students_in_course_query, conn)
display(df_students_in_course)
```

Students enrolled in CourseID 201:

	FirstName	LastName
0	John	Doe

```
# Query to count total enrollments per course
print("Total enrollments per course:")
enrollments_per_course_query = '''
SELECT c.CourseName, COUNT(e.EnrollmentID) AS TotalEnrollments
FROM Courses c
LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
GROUP BY c.CourseName;
'''

df_enrollments_per_course = pd.read_sql_query(enrollments_per_course_query, conn)
display(df_enrollments_per_course)
```

Total enrollments per course:

	CourseName	TotalEnrollments
0	Advanced SQL	0
1	Introduction to SQL	1

```
# Close the connection
conn.close()
print("Database connection closed.")
```

Database connection closed.

```
if __name__ == "__main__":
    print("This code is executed directly.")
else:
    print("This code is imported as a module.")
```

This code is executed directly.